Université M'Hamed Bougara Boumerdès, Faculté des Sciences, Département Informatique

Master 1 – Janvier 2012

Corrigé - TD N°4 BDA : **Mysql Procédural**

Resp. A. AIT BOUZIAD

Considérons les tables suivantes :

```
CREATE TABLE Compagnie
(       comp                    CHAR(4),
        nrue                    INTEGER(3),
        rue                     CHAR(20),
        ville                   CHAR(15)        DEFAULT 'Paris' COMMENT 'Par defaut : Paris',
        nomComp                 CHAR(15)        NOT NULL,
CONSTRAINT pk_Compagnie PRIMARY KEY(comp)
);
CREATE TABLE Pilote
(       brevet                  CHAR(6),
        nom                     CHAR(15) NOT NULL,
        nbHVol                  DECIMAL(7,2),
        compa                   CHAR(4),
        nbqualif                INTEGER(3),
        grade                   CHAR(4),
CONSTRAINT pk_Pilote PRIMARY KEY(brevet),
CONSTRAINT ck_nbHVol CHECK(nbHVol BETWEEN 0 AND 20000),
CONSTRAINT un_nom UNIQUE (nom),
CONSTRAINT fk_Pil_compa_Comp FOREIGN KEY (compa) REFERENCES Compagnie(comp)
);
CREATE TABLE TypeAvion
(       typa                    CHAR(6),
        nomtype                 VARCHAR(50),
CONSTRAINT pk_TypeAvion PRIMARY KEY(typa)
);
CREATE TABLE Qualifications
(       brevet                  CHAR(6),
        typa                    CHAR(6),
        dateexpiration          DATE,
CONSTRAINT pk_Qualifications PRIMARY KEY(brevet,typa)
);
```

exemple d'instance:

Pilote

BREVET	NOM	NBHVOL	COMP	NBQUALIF	GRADE
PL-1	P. Caboche	450	AF	3	
PL-2	G. Lebur	3400	AF	1	
PL-3	X. Leclercq	900	SING	1	

TypeAvion

TYPA	NOMTYPE
A320	Biréacteur Airbus 320
A330	Biréacteur Airbus 330
A340	Quadriréacteur Airbus 340
A380	Big-Quadriréacteur 380

Qualifications

BREVET	TYPA	EXPIRE
PL - 1	A340	22/06/05
PL - 1	A330	05/02/05
PL - 1	A320	16/01/04
PL - 2	A320	18/01/04
PL - 3	A330	22/01/06

Exercice 1 :

Ecrire une fonction EffectifsHeure(comp,heures)qui doit renvoyer le nombre de pilotes d'une compagnie donnée (premier paramètre) qui ont plus d'heures de vol que la valeur du deuxième paramètre (si aucun pilote, retourne 0). Si aucune compagnie n'est passée en paramètre (mettre NULL), le calcul inclut toutes les compagnies. Les éventuelles erreurs ne sont pas à traitées (compagnie de code inexistant, par exemple).

Solution :

```
CREATE FUNCTION bdsoutou.EffectifsHeure(
        pcomp VARCHAR(4),
        pheuresVol DECIMAL(7,2))
RETURNS SMALLINT
BEGIN
        DECLARE resultat SMALLINT;
        IF (pcomp IS NULL) THEN
                SELECT COUNT(*) INTO resultat FROM Pilote WHERE nbHVol > pheuresVol;
        ELSE
                SELECT COUNT(*) INTO resultat FROM Pilote WHERE nbHVol > pheuresVol
                                AND comp = pcomp;
        END IF;
        RETURN resultat;
END;
```

Exercice 2 :

Ecrire une procédure PlusExperimente(comp,nom,heures) qui doit retourner le nom et le nombre d'heures de vol du pilote (par l'intermédiaire des deuxième et troisième paramètres) le plus expérimenté d'une compagnie donnée (premier paramètre). Si plusieurs pilotes ont la même expérience, un message d'erreur est affiché. Si aucune compagnie n'est passée en paramètre (mettre NULL), la procédure retourne le nom du plus expérimenté et le code de sa compagnie (par l'intermédiaire du premier paramètre).

Solution :

```
CREATE PROCEDURE bdsoutou.PlusExperimente
 (INOUT pcomp VARCHAR(4), OUT pnomPil VARCHAR(20), OUT pheuresVol DECIMAL(7,2))
BEGIN
 DECLARE p1 SMALLINT;
 IF (pcomp IS NULL) THEN
   SELECT COUNT(*) INTO p1 FROM Pilote
          WHERE nbHVol=(SELECT MAX(nbHVol) FROM Pilote);
 ELSE
   SELECT COUNT(*) INTO p1 FROM Pilote
          WHERE nbHVol=(SELECT MAX(nbHVol) FROM Pilote WHERE comp=pcomp)
          AND comp = pcomp;
 END IF;
 IF (p1 = 0) THEN
   SELECT ('Aucun pilote n''est le plus expérimenté') AS resultat;
 ELSEIF p1 > 1 THEN
  SELECT('Plusieurs pilotes sont les plus expérimentés') AS resultat;
 ELSE
   IF (pcomp IS NULL) THEN
       SELECT nom, nbHVol, comp INTO pnomPil, pheuresVol, pcomp
        FROM Pilote WHERE nbHVol=(SELECT MAX(nbHVol) FROM Pilote);
   ELSE
       SELECT nom, nbHVol INTO pnomPil, pheuresVol FROM Pilote
           WHERE nbHVol=(SELECT MAX(nbHVol) FROM Pilote WHERE comp=pcomp)
           AND comp = pcomp;
   END IF;
 END IF;
END;
```

Exercice 3 :

Modifier la fonction EffectifsHeure(comp,heures) en procédure de telle sorte à donner les mêmes informations

Exercice 4 :

Ecrire une fonction récursive Sql qui calcul la factorielle d'un entier n. Ensuite tester cette fonction pour une valeur donnée.

Solution:

1)

```
delimiter $
CREATE FUNCTION factorielle(n INT) RETURNS INT
BEGIN
        IF n = 1 THEN
                RETURN (1);
        ELSE
                RETURN (n * factorielle(n - 1));
        END IF;
END;
$
```

```
2)  SELECT factorielle de 10 AS 'Factorielle de 10'$
Réponse attendue de Mysql :
ERROR 1424 (HY000): Recursive stored routines are not allowed.
```

Exercice 5 :

Ecrire une procédure Sql qui calcul la somme des heures de vol de tous les pilotes de la compagnie de code 'AF'.
Utiliser un curseur.

Solution :

Delimiter $

```
DECLARE  procedure total_hvol_AF()
BEGIN
        DECLARE fincurs1 BOOLEAN DEFAULT 0;
        DECLARE v_nbHv DECIMAL(7,2);
        DECLARE v_tot DECIMAL(8,2) DEFAULT 0;
        DECLARE curs1 CURSOR FOR
                SELECT nbHVol
                FROM bdsoutou.Pilote
                WHERE comp = 'AF';
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET fincurs1 := 1;

        /* Ouverture du curseur */
```

```
        OPEN curs1;

        /* Recherche d'une ligne */
        FETCH curs1 INTO v_nbHv;

        /* Calcul de la somme */
        WHILE (NOT fincurs1)
        DO
                SET v_tot := v_tot+v_nbHv;
                FETCH curs1 INTO v_nbHv;
        END WHILE;

        /* Fermeture du curseur */
        CLOSE curs1;

        SELECT v_tot AS 'Total heures pour les pilotes de ''AF''';
END ;
$
delimiter ;
call total_hvol_AF() ;
```

Exercice 6 :

Ecrire une procédure utilisant un curseur pour :

- augmenter le nombre d'heures de 100 pour les pilotes de la compagnie de code 'AF' ;
- diminuer ce nombre de 100 pour les pilotes de la compagnie de code 'SING' ;
- supprimer les pilotes des autres compagnies.

Verrouiller les lignes à modifiées !

Solution :

Delimiter $

```
DECLARE procedure exo6()
BEGIN
        DECLARE fincurs BOOLEAN DEFAULT 0;
        DECLARE v_brevet VARCHAR(6);
        DECLARE v_nbHv DECIMAL(7,2);
        DECLARE v_comp VARCHAR(4);
        DECLARE curs CURSOR FOR
                SELECT brevet,nbHVol,comp
                FROM Pilote FOR UPDATE;
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET fincurs := 1;

        SET AUTOCOMMIT = 0;

        OPEN curs;
        FETCH curs INTO v_brevet,v_nbHv,v_comp;

        WHILE (NOT fincurs) DO
                IF (v_comp='AF') THEN
                                UPDATE Pilote
                                SET nbHVol = nbHVol + 100
                                WHERE brevet = v_brevet;
                ELSEIF (v_comp='SING') THEN
                                UPDATE bdsoutou.Pilote
                                SET nbHVol = nbHVol - 100
                                WHERE brevet = v_brevet;
                ELSE
                                DELETE FROM bdsoutou.Pilote
                                WHERE brevet = v_brevet;
                END IF;

                FETCH curs INTO v_brevet,v_nbHv,v_comp;
        END WHILE;

        CLOSE curs;
        COMMIT;
END;
$
Delimiter;
CALL exo6();
```

Commentaire :

- Si vous voulez verrouiller les lignes d'une table interrogée par un curseur dans le but de mettre à jour la table, sans qu'un autre utilisateur ne la modifie en même temps, il faut utiliser la clause FOR UPDATE. Elle s'emploie lors de la déclaration du curseur et verrouille les lignes concernées dès l'ouverture du curseur. Les verrous sont libérés à la fin de la transaction.

- Une validation (COMMIT) avant la fermeture d'un curseur FOR UPDATE aura des effets de bord fâcheux.

- Il n'est pas possible de déclarer un curseur FOR UPDATE en utilisant dans la requête les directives DISTINCT ou GROUP BY, un opérateur ensembliste, ou une fonction d'agrégat.

- Il n'existe pas encore de directive WHERE CURRENT OF pour modifier l'enregistrement courant d'un curseur avec verrou.

- Un curseur, comme un tableau, ne peut pas être passé en paramètre d'un sous-programme ni en entrée (IN), ni en sortie (OUT).

Exercice 7 :

Etant donnée règle de gestion suivante:

tout pilote qui perd une qualification doit voir son compteur automatiquement décrémenter.

Programmer le déclencheur `TrigDelQualif` qui surveille les suppressions de la table `Qualifications`, et diminue de un la colonne `nbqualif` pour le pilote concerné par la suppression de sa qualification.

Solution:

```
CREATE TRIGGER TriDelQualif
       AFTER DELETE ON Qualifications
FOR EACH ROW
BEGIN
       UPDATE Pilote SET nbQualif = nbQualif - 1
       WHERE brevet = OLD.brevet;
END;

Testons :
DELETE FROM Qualifications WHERE typa = 'A320';
```

Exercice 8 :

Etant donnée règle de gestion suivante:

tout pilote qui gagne une qualification doit voir son compteur automatiquement incrémenter.

Programmer le déclencheur `TrigInsQualif` qui surveille les insertions sur la table Qualifications et augmente de un la colonne `nbqualif` pour le pilote concerné.

Solution:

```
CREATE TRIGGER TrigInsQualif
       AFTER INSERT ON Qualifications
FOR EACH ROW
BEGIN
       UPDATE Pilote SET nbQualif = nbQualif + 1
              WHERE brevet = NEW.brevet;
END;

Testons:
INSERT INTO Qualifications VALUES ('PL-2', 'A380',SYSDATE());
Puis :
SELECT * FROM Qualifications ; et
SELECT * FROM Pilote ;
```

Exercice 9 :

Dans le cas de la modification de la colonne brevet d'un enregistrement de la table Qualifications.

Exemple :

```
       UPDATE Qualifications
       SET brevet = 'PL-2'
       WHERE brevet = 'PL-3'AND typa = 'A330';
```

Programmer le déclencheur qui doit permettre de garder la cohérence de la colonne nbquali dans la table pilotes.

Solution:

```
CREATE TRIGGER TrigUpdQualif
       AFTER UPDATE ON Qualifications
FOR EACH ROW
BEGIN
       UPDATE Pilote
       SET nbQualif = nbQualif + 1
       WHERE brevet = NEW.brevet;

       UPDATE Pilote
       SET nbQualif = nbQualif - 1
       WHERE brevet = OLD.brevet;
END;

Testons:
```

```
UPDATE Qualifications
SET brevet = 'PL-2'
WHERE brevet = 'PL-3'AND typa = 'A330';

SELECT * FROM Pilote ;
SELECT * FROM Qualifications
```

Exercice 9 :

Etant donnée règle de gestion suivante: (contraintes de domaine)

Un pilote

- ne peut être commandant de bord qu'à la condition qu'il ait entre 1 000 et 4 000 heures de Vol
- ne peut être copilote qu'à la condition qu'il ait entre 100 et 1 000 heures de vol
- ne peut être instructeur qu'à partir de 3 000 heures de vol.

Programmer le déclencheur `TrigInsGrade` qui surveille les insertions sur la table et forcer la valeur de la colonne `grade` pour conserver la cohérence de la table pilote.

Solution:

```
CREATE TRIGGER TrigInsGrade
       BEFORE INSERT ON Pilote FOR EACH ROW
BEGIN
IF (NEW.grade = 'CDB' AND (NEW.nbHVol <1000)) THEN
       SET NEW.grade:= 'COPI';
END IF;
IF (NEW.grade = 'CDB' AND (NEW.nbHVol >4000)) THEN
       SET NEW.grade:= 'INST';
END IF;
IF (NEW.grade = 'COPI' AND (NEW.nbHVol >1000)) THEN
       SET NEW.grade:= 'CDB';
END IF;
IF (NEW.grade = 'INST' AND (NEW.nbHVol <3000)) OR (NEW.nbHVol <100) THEN
       SET NEW.grade:= NULL;
END IF;
END;
```

Testons :

```
INSERT INTO Pilote VALUES
('PL-1','Daniel Vielle',1000,'CDB');
INSERT INTO Pilote VALUES
('PL-2','Benoit Treihlou',450,'COPI');
INSERT INTO Pilote VALUES
('PL-3','Pierre Filoux',9000,'INST');
INSERT INTO Pilote VALUES
('PL-4','Philippe Minier',1000,'COPI');
INSERT INTO Pilote VALUES
('PL-5','Trop jeune', 100,'CDB' );
INSERT INTO Pilote VALUES
('PL-6','Inexperimente', 2999,'INST');
```

```
SELECT * FROM Pilote;
+------+-----------------+---------+-------+
| breve| nom             | nbHVol  | grade |
+------+-----------------+---------+-------+
| PL-1 | Daniel Vielle   | 1000.00 | CDB   |
| PL-2 | Benoit Treihlou | 450.00  | COPI  |
| PL-3 | Pierre Filoux   | 9000.00 | INST  |
| PL-4 | Philippe Minier | 1000.00 | COPI  |
| PL-5 | Trop jeune      | 100.00  | COPI  |
| PL-6 | Inexperimente   | 2999.00 | NULL  |
+--------+---------------+---------+-------+
```

Exercice 10:

Considérons la contrainte que « tout pilote ne peut être qualifié sur plus de trois types d'appareils ». Ici, il s'agit d'assurer la cohérence entre la valeur de la colonne nbQualif de la table Pilote et les enregistrements de la table Qualifications en cas d'insertion du type :

```
INSERT INTO Qualifications VALUES ('PL-3' ,'A380',SYSDATE())
```

Programmer le déclencheur qui garanti la cohérence de la base vis-à-vis de cette règle.

Solution:
On crée d'abord la table trace :

```
CREATE TABLE Trace(col VARCHAR(80) PRIMARY KEY).
```

Ensuite:

```
CREATE TRIGGER TrigInsQualif
        BEFORE INSERT ON Qualifications
        FOR EACH ROW
BEGIN
        DECLARE v_compteur TINYINT(1);
        DECLARE v_nom VARCHAR(30);
        SELECT nbQualif, nom INTO v_compteur, v_nom
                FROM Pilote WHERE brevet = NEW.brevet;

        IF v_compteur < 3 THEN
                UPDATE Pilote
                SET nbQualif = nbQualif + 1
                WHERE brevet = NEW.brevet;
         ELSE
                INSERT INTO TRACE VALUES
                                (CONCAT('Le pilote ',v_nom, ' a déjà 3
                        qualifications!'));
                INSERT INTO TRACE VALUES (NULL);
        END IF;
END;

Et si on fait un ajout incorrect :
INSERT INTO Qualifications VALUES ('PL-1','A380', SYSDATE())$

ERROR 1048 (23000): Column 'col' cannot be null
-- ne fait pas l'INSERT dans Qualifications
-- ni dans Trace !
```